

部署及驗證 GKE NFO 多網路與高效能介面

來源：<https://codelabs.developers.google.com/codelabs/gke-multinic-l3netdevice?hl=zh-tw>

步驟數：17

在本程式碼研究室中，您將瞭解如何設定及驗證 GKE L3 和網路多裝置節點集區。

目錄

1. [簡介](#)
2. [術語和概念](#)
3. [設定主要虛擬私有雲](#)
4. [建立 GKE 叢集](#)
5. [netdevice-vpc 設定](#)
6. [I3-vpc 設定](#)
7. [建立多重 NIC 節點集區](#)
8. [驗證 multnic-node-pool](#)
9. [建立 netdevice-network](#)
10. [建立 L3 網路](#)
11. [建立 netdevice-I3-pod](#)
12. [建立 I3-pod](#)
13. [防火牆更新](#)
14. [驗證 Pod 連線](#)
15. [防火牆記錄](#)
16. [清理](#)
17. [恭喜](#)

步驟 1 · 約 0 分鐘

部署及驗證 GKE NFO 多網路和高效能介面

程式碼研究室簡介

subject上次更新時間：3月 20, 2026

account_circle作者：Deepak Michael

1. 簡介

GCP 長期以來支援 VM 執行個體層級的多個介面。有了多個介面，VM 最多可將 7 個新介面 (預設 + 7 個介面) 連至不同的 VPC。GKE 網路現在會將這項行為擴展至節點上執行的 Pod。在此功能推出前，GKE 叢集只允許所有 NodePool 擁有單一介面，因此會對應至單一 VPC。透過 Pod 的多網路功能，使用者現在可以在 GKE 叢集的節點和 Pod 中啟用多個介面。

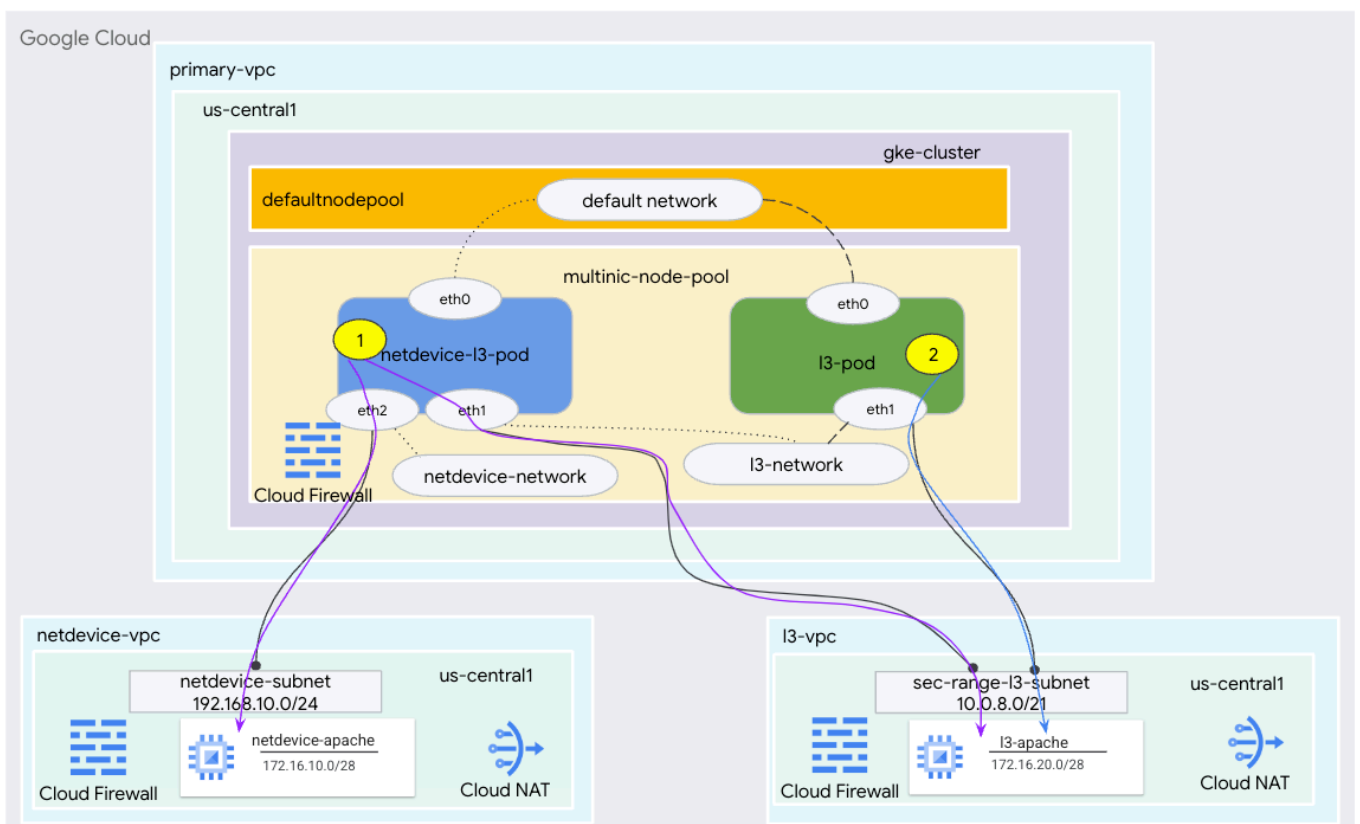
建構項目

在本教學課程中，您將建構完整的 GKE 多重 NIC 環境，說明圖 1 所示的用途。

1. 使用 BusyBox 建立 netdevice-l3-pod，以便：
2. 透過 eth2 對 netdevice-vpc 中的 netdevice-apache 執行個體執行 PING 和 wget -S
3. 透過 eth1 對 l3-vpc 中的 l3-apache 執行個體執行 PING 和 wget -S
4. 建立利用 busybox 執行 PING 和 wget -S 的 l3-pod，透過 eth1 連線至 l3-apache 執行個體

在這兩種情況下，Pod 的 eth0 介面都會連線至預設網路。

圖 1



注意：Codelab 會根據圖示拓撲提供設定和驗證步驟，請視需要修改程序，以符合貴機構的需求。

課程內容

- 如何建立 L3 類型子網路
- 如何建立 netdevice 類型子網路
- 如何建立多 NIC GKE 節點集區
- 如何建立具備 netdevice 和 L3 功能的 Pod
- 如何建立具有 L3 功能的 Pod
- 如何建立及驗證 GKE 物件網路
- 如何使用 PING、wget 和防火牆記錄，驗證與遠端 Apache 伺服器的連線

軟硬體需求

- Google Cloud 專案
-

2. 術語和概念

主要虛擬私有雲：主要 **VPC** 是預先設定的 VPC，內含一組預設設定和資源。GKE 叢集會在這個虛擬私有雲中建立。

子網路：在 Google Cloud 中，**子網路**是在虛擬私有雲中建立無類別跨網域路由 (CIDR) 和網路遮罩的方式。子網路有一個主要 IP 位址範圍 (指派給節點)，以及多個可屬於 Pod 和 Service 的次要範圍。

節點網路：節點網路是指 VPC 和子網路配對的專屬組合。在這個節點網路中，節點集區的節點會從主要 IP 位址範圍分配 IP 位址。

次要範圍：Google Cloud 次要範圍是屬於 VPC 中某個區域的 CIDR 和網路遮罩。GKE 會將此網路做為第 3 層 Pod 網路。一個 VPC 可以有多個次要範圍，而一個 Pod 可以連線至多個 Pod 網路。

網路 (第 3 層或裝置)：做為 Pod 連線點的網路物件。在本教學課程中，網路為 `I3-network` 和 `netdevice-network`，裝置可以是 `netdevice` 或 `dpdk`。預設網路為必要項目，會在叢集建立時，根據預設節點集區子網路建立。

第 3 層網路對應於子網路上的次要範圍，表示方式如下：

虛擬私有雲 -> 子網路名稱 -> 次要範圍名稱

裝置網路對應至虛擬私有雲上的子網路，表示方式如下：

VPC -> 子網路名稱

預設 Pod 網路：Google Cloud 會在建立叢集時建立預設 Pod 網路。預設 Pod 網路會使用主要 VPC 做為節點網路。根據預設，所有叢集節點和 Pod 都可使用預設的 Pod 網路。

具有多個介面的 Pod：GKE 中具有多個介面的 Pod 無法連線至相同的 Pod 網路，因為 Pod 的每個介面都必須連線至專屬網路。

更新專案以支援程式碼研究室

本程式碼研究室會使用 `$variables`，協助您在 Cloud Shell 中實作 `gcloud` 設定。

在 Cloud Shell 中執行下列操作：

□□□□

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

步驟 3 · 約 10 分鐘

3. 設定主要虛擬私有雲

建立主要虛擬私有雲

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create primary-vpc --project=$projectid --subnet-mode=custom
```

建立節點和次要子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create primary-node-subnet --project=$projectid --  
range=192.168.0.0/24 --network=primary-vpc --region=us-central1 --enable-private-ip-google-  
access --secondary-range=sec-range-primay-vpc=10.0.0.0/21
```

步驟 4 · 約 10 分鐘

4. 建立 GKE 叢集

建立私人 GKE 叢集，並指定主要虛擬私有雲子網路，以使用必要的標記 (`--enable-multi-networking` 和 `--enable-dataplane-v2`) 建立預設節點集區，支援多 NIC 節點集區。

在 Cloud Shell 中建立 GKE 叢集：

```
gcloud container clusters create multinic-gke \
  --zone "us-central1-a" \
  --enable-dataplane-v2 \
  --enable-ip-alias \
  --enable-multi-networking \
  --network "primary-vpc" --subnetwork "primary-node-subnet" \
  --num-nodes=2 \
  --max-pods-per-node=32 \
  --cluster-secondary-range-name=sec-range-primay-vpc \
  --no-enable-master-authorized-networks \
  --release-channel "regular" \
  --enable-private-nodes --master-ipv4-cidr "100.100.10.0/28" \
  --enable-ip-alias
```

驗證 multinic-gke 叢集

在 Cloud Shell 中，使用叢集進行驗證：

```
gcloud container clusters get-credentials multinic-gke --zone us-central1-a --project
$projectid
```

在 Cloud Shell 中，確認已從 default-pool 產生兩個節點：

```
kubectl get nodes
```

範例：

```
user@$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
gke-multinic-gke-default-pool-3d419e48-1k2p	Ready	<none>	2m4s	v1.27.3-gke.100
gke-multinic-gke-default-pool-3d419e48-xckb	Ready	<none>	2m4s	v1.27.3-gke.100

步驟 5 · 約 5 分鐘

5. netdevice-vpc 設定

建立 netdevice-vpc 網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create netdevice-vpc --project=$projectid --subnet-mode=custom
```

建立 netdevice-vpc 子網路

在 Cloud Shell 中，建立用於 multinic netdevice 網路的子網路：

```
gcloud compute networks subnets create netdevice-subnet --project=$projectid --range=192.168.10.0/24 --network=netdevice-vpc --region=us-central1 --enable-private-ip-google-access
```

在 Cloud Shell 中，為 netdevice-apache 執行個體建立子網路：

```
gcloud compute networks subnets create netdevice-apache --project=$projectid --range=172.16.10.0/28 --network=netdevice-vpc --region=us-central1 --enable-private-ip-google-access
```

Cloud Router 和 NAT 設定

由於 VM 執行個體沒有外部 IP 位址，因此本教學課程會使用 Cloud NAT 安裝軟體套件。

在 Cloud Shell 中建立 Cloud Router。

```
gcloud compute routers create netdevice-cr --network netdevice-vpc --region us-central1
```

在 Cloud Shell 中建立 NAT 閘道。

```
gcloud compute routers nats create cloud-nat-netdevice --router=netdevice-cr --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges --region us-central1
```

建立 netdevice-apache 執行個體

在下一節中，您將建立 netdevice-apache 執行個體。

在 Cloud Shell 中建立執行個體：


```
gcloud compute instances create netdevice-apache \  
  --project=$projectid \  
  --machine-type=e2-micro \  
  --image-family debian-11 \  
  --no-address \  
  --image-project debian-cloud \  
  --zone us-central1-a \  
  --subnet=netdevice-apache \  
  --metadata startup-script="#! /bin/bash  
    sudo apt-get update  
    sudo apt-get install apache2 -y  
    sudo service apache2 restart  
    echo 'Welcome to the netdevice-apache instance !!' | tee /var/www/html/index.html  
    EOF"
```

步驟 6 · 約 5 分鐘

6. I3-vpc 設定

建立 I3-vpc 網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create l3-vpc --project=$projectid --subnet-mode=custom
```

建立 I3-vpc 子網路

在 Cloud Shell 中，建立主要和次要範圍的子網路。次要範圍(sec-range-l3-subnet) 用於 multinet I3 網路：

```
gcloud compute networks subnets create l3-subnet --project=$projectid --range=192.168.20.0/24  
--network=l3-vpc --region=us-central1 --enable-private-ip-google-access --secondary-  
range=sec-range-l3-subnet=10.0.8.0/21
```

在 Cloud Shell 中，為 I3-apache 執行個體建立子網路：

```
gcloud compute networks subnets create l3-apache --project=$projectid --range=172.16.20.0/28  
--network=l3-vpc --region=us-central1 --enable-private-ip-google-access
```

Cloud Router 和 NAT 設定

由於 VM 執行個體沒有外部 IP 位址，因此本教學課程會使用 Cloud NAT 安裝軟體套件。

在 Cloud Shell 中建立 Cloud Router。

```
gcloud compute routers create l3-cr --network l3-vpc --region us-central1
```

在 Cloud Shell 中建立 NAT 閘道。

```
gcloud compute routers nats create cloud-nat-l3 --router=l3-cr --auto-allocate-nat-external-  
ips --nat-all-subnet-ip-ranges --region us-central1
```

建立 I3-apache 執行個體

在下一節中，您將建立 I3-apache 執行個體。

在 Cloud Shell 中建立執行個體：

```
gcloud compute instances create l3-apache \  
  --project=$projectid \  
  --machine-type=e2-micro \  
  --image-family debian-11 \  
  --no-address \  
  --image-project debian-cloud \  
  --zone us-central1-a \  
  --subnet=l3-apache \  
  --metadata startup-script="#! /bin/bash  
    sudo apt-get update  
    sudo apt-get install apache2 -y  
    sudo service apache2 restart  
    echo 'Welcome to the l3-apache instance !!' | tee /var/www/html/index.html  
  EOF"
```

步驟 7 · 約 2 分鐘

7. 建立多重 NIC 節點集區

在下一節中，您將建立包含下列旗標的 multinic 節點集區：

`--additional-node-network` (裝置類型介面必須提供)

範例：

```
--additional-node-network network=netdevice-vpc,subnetwork=netdevice-subnet
```

`--additional-node-network` 和 `--additional-pod-network` (L3 類型介面必須使用)

範例：

```
--additional-node-network network=l3-vpc,subnetwork=l3-subnet --additional-pod-network  
subnetwork=l3-subnet,pod-ipv4-range=sec-range-l3-subnet,max-pods-per-node=8
```

機器類型：部署節點集區時，請考慮機器類型依附元件。舉例來說，如果機器類型為「e2-standard-4」，具有 4 個 vCPU，則最多可支援 4 個 VPC。舉例來說，netdevice-l3-pod 總共會有 3 個介面 (預設、netdevice 和 l3)，因此本教學課程使用的機器類型為 e2-standard-4。

在 Cloud Shell 中，建立由 Type Device 和 L3 組成的節點集區：

```
gcloud container --project "$projectid" node-pools create "multinic-node-pool" --cluster  
"multinic-gke" --zone "us-central1-a" --additional-node-network network=netdevice-  
vpc,subnetwork=netdevice-subnet --additional-node-network network=l3-vpc,subnetwork=l3-subnet  
--additional-pod-network subnetwork=l3-subnet,pod-ipv4-range=sec-range-l3-subnet,max-pods-  
per-node=8 --machine-type "e2-standard-4"
```

8. 驗證 multnic-node-pool

在 Cloud Shell 中，驗證是否從 multinic-node-pool 產生三個節點：

```
kubectl get nodes
```

範例：

```
user@$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
gke-multinic-gke-default-pool-3d419e48-1k2p	Ready	<none>	15m	v1.27.3-gke.100
gke-multinic-gke-default-pool-3d419e48-xckb	Ready	<none>	15m	v1.27.3-gke.100
gke-multinic-gke-multinic-node-pool-135699a1-0tfx	Ready	<none>	3m51s	v1.27.3-gke.100
gke-multinic-gke-multinic-node-pool-135699a1-86gz	Ready	<none>	3m51s	v1.27.3-gke.100
gke-multinic-gke-multinic-node-pool-135699a1-t66p	Ready	<none>	3m51s	v1.27.3-gke.100

步驟 9 · 約 10 分鐘

9. 建立 netdevice-network

在下列步驟中，您將產生 Network 和 GKENetworkParamSet Kubernetes 物件，以建立 netdevice 網路，用於在後續步驟中關聯 Pod。

建立 netdevice-network 物件

在 Cloud Shell 中，使用 VI 編輯器或 nano 建立網路物件 YAML 檔案 netdevice-network.yaml。請注意，「routes to」是 netdevice-vpc 中的子網路 172.16.10.0/28 (netdevice-apache)。

```
apiVersion: networking.gke.io/v1
kind: Network
metadata:
  name: netdevice-network
spec:
  type: "Device"
  parametersRef:
    group: networking.gke.io
    kind: GKENetworkParamSet
    name: "netdevice"
  routes:
    - to: "172.16.10.0/28"
```

在 Cloud Shell 中套用 netdevice-network.yaml：

```
kubectl apply -f netdevice-network.yaml
```

在 Cloud Shell 中，驗證 netdevice-network 狀態類型是否為「Ready」。

```
kubectl describe networks netdevice-network
```

範例：

```
user@$ kubectl describe networks netdevice-network
Name:          netdevice-network
Namespace:
Labels:        <none>
Annotations:   networking.gke.io/in-use: false
API Version:   networking.gke.io/v1
Kind:          Network
Metadata:
  Creation Timestamp:  2023-07-30T22:37:38Z
  Generation:         1
  Resource Version:    1578594
  UID:                46d75374-9fcc-42be-baeb-48e074747052
Spec:
  Parameters Ref:
    Group:  networking.gke.io
    Kind:   GKENetworkParamSet
    Name:   netdevice
  Routes:
    To:  172.16.10.0/28
  Type:  Device
Status:
  Conditions:
    Last Transition Time:  2023-07-30T22:37:38Z
    Message:              GKENetworkParamSet resource was deleted: netdevice
    Reason:               GNPDeleted
    Status:               False
    Type:                 ParamsReady
    Last Transition Time:  2023-07-30T22:37:38Z
    Message:              Resource referenced by params is not ready
    Reason:               ParamsNotReady
    Status:               False
    Type:                 Ready
  Events:                 <none>
```

建立 GKENetworkParamSet

在 Cloud Shell 中，使用 VI 編輯器或 nano 建立網路物件 YAML 檔案 netdevice-network-param.yaml。規格會對應至 netdevice-vpc 子網路部署作業。

```
apiVersion: networking.gke.io/v1
kind: GKENetworkParamSet
metadata:
  name: "netdevice"
spec:
  vpc: "netdevice-vpc"
  vpcSubnet: "netdevice-subnet"
  deviceMode: "NetDevice"
```

在 Cloud Shell 中套用 netdevice-network-param.yaml

```
kubectl apply -f netdevice-network-parm.yaml
```

在 Cloud Shell 中，驗證 netdevice-network 狀態原因 GNPParmsReady 和 NetworkReady：

```
kubectl describe networks netdevice-network
```

範例：

```
user@$ kubectl describe networks netdevice-network
Name:          netdevice-network
Namespace:
Labels:        <none>
Annotations:   networking.gke.io/in-use: false
API Version:   networking.gke.io/v1
Kind:          Network
Metadata:
  Creation Timestamp:  2023-07-30T22:37:38Z
  Generation:         1
  Resource Version:    1579791
  UID:                46d75374-9fcc-42be-baeb-48e074747052
Spec:
  Parameters Ref:
    Group:  networking.gke.io
    Kind:   GKENetworkParamSet
    Name:   netdevice
  Routes:
    To:  172.16.10.0/28
  Type:  Device
Status:
  Conditions:
    Last Transition Time:  2023-07-30T22:39:44Z
    Message:
    Reason:                GNPParmsReady
    Status:                True
    Type:                  ParamsReady
    Last Transition Time:  2023-07-30T22:39:44Z
    Message:
    Reason:                NetworkReady
    Status:                True
    Type:                  Ready
  Events:                  <none>
```

在 Cloud Shell 中，驗證後續步驟中用於 Pod 介面的 gkenetworkparamset CIDR 區塊 192.168.10.0/24。

```
kubectl describe gkenetworkparamsets.networking.gke.io netdevice
```

範例：


```
user@$ kubectl describe gkenetworkparamsets.networking.gke.io netdevice
```

Name: netdevice

Namespace:

Labels: <none>

Annotations: <none>

API Version: networking.gke.io/v1

Kind: GKENetworkParamSet

Metadata:

Creation Timestamp: 2023-07-30T22:39:43Z

Finalizers:

networking.gke.io/gnp-controller

networking.gke.io/high-perf-finalizer

Generation: 1

Resource Version: 1579919

UID: 6fe36b0c-0091-4b6a-9d28-67596cbce845

Spec:

Device Mode: NetDevice

Vpc: netdevice-vpc

Vpc Subnet: netdevice-subnet

Status:

Conditions:

Last Transition Time: 2023-07-30T22:39:43Z

Message:

Reason: GNPRReady

Status: True

Type: Ready

Network Name: netdevice-network

Pod CID Rs:

Cidr Blocks:

192.168.10.0/24

Events: <none>

步驟 10 · 約 10 分鐘

10. 建立 L3 網路

在下列步驟中，您將產生 Kubernetes 物件的網路和 GKENetworkParamSet，以建立 L3 網路，用於在後續步驟中關聯 Pod。

建立 I3 網路物件

在 Cloud Shell 中，使用 VI 編輯器或 nano 建立網路物件 YAML l3-network.yaml。請注意，「routes to」是 l3-vpc 中的子網路 172.16.20.0/28 (l3-apache)。

```
apiVersion: networking.gke.io/v1
kind: Network
metadata:
  name: l3-network
spec:
  type: "L3"
  parametersRef:
    group: networking.gke.io
    kind: GKENetworkParamSet
    name: "l3-network"
  routes:
    - to: "172.16.20.0/28"
```

在 Cloud Shell 中套用 l3-network.yaml：

```
kubectl apply -f l3-network.yaml
```

在 Cloud Shell 中，驗證 l3-network 狀態類型是否為「Ready」。

```
kubectl describe networks l3-network
```

範例：

```

user@$ kubectl describe networks l3-network
Name:          l3-network
Namespace:
Labels:        <none>
Annotations:   networking.gke.io/in-use: false
API Version:   networking.gke.io/v1
Kind:          Network
Metadata:
  Creation Timestamp:  2023-07-30T22:43:54Z
  Generation:         1
  Resource Version:    1582307
  UID:                426804be-35c9-4cc5-bd26-00b94be2ef9a
Spec:
  Parameters Ref:
    Group:  networking.gke.io
    Kind:   GKENetworkParamSet
    Name:   l3-network
  Routes:
    to:  172.16.20.0/28
    Type: L3
Status:
  Conditions:
    Last Transition Time:  2023-07-30T22:43:54Z
    Message:              GKENetworkParamSet resource was deleted: l3-network
    Reason:               GNPDeleted
    Status:               False
    Type:                 ParamsReady
    Last Transition Time:  2023-07-30T22:43:54Z
    Message:              Resource referenced by params is not ready
    Reason:               ParamsNotReady
    Status:               False
    Type:                 Ready
Events:                  <none>

```

建立 GKENetworkParamSet

在 Cloud Shell 中，使用 VI 編輯器或 nano 建立網路物件 YAML l3-network-param.yaml。請注意，規格會對應至 l3-vpc 子網路部署作業。

```

apiVersion: networking.gke.io/v1
kind: GKENetworkParamSet
metadata:
  name: "l3-network"
spec:
  vpc: "l3-vpc"
  vpcSubnet: "l3-subnet"
  podIPv4Ranges:
    rangeNames:
      - "sec-range-l3-subnet"

```

在 Cloud Shell 中套用 l3-network-parm.yaml

```
kubectl apply -f l3-network-parm.yaml
```

在 Cloud Shell 中，驗證 l3-network 的「狀態原因」為 GNPParmsReady 和 NetworkReady：

```
kubectl describe networks l3-network
```

範例：

```
user@$ kubectl describe networks l3-network
Name:          l3-network
Namespace:
Labels:        <none>
Annotations:   networking.gke.io/in-use: false
API Version:   networking.gke.io/v1
Kind:          Network
Metadata:
  Creation Timestamp:  2023-07-30T22:43:54Z
  Generation:         1
  Resource Version:    1583647
  UID:                426804be-35c9-4cc5-bd26-00b94be2ef9a
Spec:
  Parameters Ref:
    Group:  networking.gke.io
    Kind:   GKENetworkParamSet
    Name:   l3-network
  Routes:
    To:  172.16.20.0/28
  Type:  L3
Status:
  Conditions:
    Last Transition Time:  2023-07-30T22:46:14Z
    Message:
    Reason:                GNPParmsReady
    Status:                True
    Type:                  ParamsReady
    Last Transition Time:  2023-07-30T22:46:14Z
    Message:
    Reason:                NetworkReady
    Status:                True
    Type:                  Ready
  Events:                  <none>
```

在 Cloud Shell 中，驗證用於建立 Pod 介面的 gkenetworkparamset l3 網路 CIDR 10.0.8.0/21。

```
kubectl describe gkenetworkparamsets.networking.gke.io l3-network
```

範例：

```
user@$ kubectl describe gkenetworkparamsets.networking.gke.io l3-network
```

Name: l3-network

Namespace:

Labels: <none>

Annotations: <none>

API Version: networking.gke.io/v1

Kind: GKENetworkParamSet

Metadata:

Creation Timestamp: 2023-07-30T22:46:14Z

Finalizers:

networking.gke.io/gnp-controller

Generation: 1

Resource Version: 1583656

UID: 4c1f521b-0088-4005-b000-626ca5205326

Spec:

podIPv4Ranges:

Range Names:

sec-range-l3-subnet

Vpc: l3-vpc

Vpc Subnet: l3-subnet

Status:

Conditions:

Last Transition Time: 2023-07-30T22:46:14Z

Message:

Reason: GNPRReady

Status: True

Type: Ready

Network Name: l3-network

Pod CID Rs:

Cidr Blocks:

10.0.8.0/21

Events: <none>

11. 建立 netdevice-l3-pod

在下一節中，您將建立執行 busybox 的 netdevice-l3-pod，這項工具支援超過 300 個常見指令，因此又稱為「瑞士刀」。Pod 已設定為使用 eth1 與 l3-vpc 通訊，並使用 eth2 與 netdevice-vpc 通訊。

在 Cloud Shell 中，使用 VI 編輯器或 nano 建立名為 netdevice-l3-pod.yaml 的 BusyBox 容器。

```
apiVersion: v1
kind: Pod
metadata:
  name: netdevice-l3-pod
  annotations:
    networking.gke.io/default-interface: 'eth0'
    networking.gke.io/interfaces: |
      [
        {"interfaceName": "eth0", "network": "default"},
        {"interfaceName": "eth1", "network": "l3-network"},
        {"interfaceName": "eth2", "network": "netdevice-network"}
      ]
spec:
  containers:
    - name: netdevice-l3-pod
      image: busybox
      command: ["sleep", "10m"]
      ports:
        - containerPort: 80
      restartPolicy: Always
```

在 Cloud Shell 中套用 netdevice-l3-pod.yaml

```
kubectl apply -f netdevice-l3-pod.yaml
```

驗證 netdevice-l3-pod 建立作業

在 Cloud Shell 中，驗證 netdevice-l3-pod 是否正在執行：

```
kubectl get pods netdevice-l3-pod
```

範例：

```
user@$ kubectl get pods netdevice-l3-pod
NAME                READY   STATUS    RESTARTS   AGE
netdevice-l3-pod    1/1     Running   0           74s
```

在 Cloud Shell 中，驗證指派給 Pod 介面的 IP 位址。

```
kubectl get pods netdevice-l3-pod -o yaml
```

在提供的範例中，networking.gke.io/pod-ips 欄位包含與 I3-network 和 netdevice-network 中 Pod 介面相關聯的 IP 位址。預設網路 IP 位址 10.0.1.22 的詳細資料位於 podIPs 下方：

```

user@$ kubectl get pods netdevice-l3-pod -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"networking.gke.io/default-
interface":"eth0","networking.gke.io/interfaces":
[\n{"interfaceName":"eth0","network":"default"},\n{"interfaceName":"eth1","netwo
rk":"l3-network"},\n{"interfaceName":"eth2","network":"netdevice-
network"}\n]\n},"name":"netdevice-l3-pod","namespace":"default"},"spec":{"containers":
[{"command":["sleep","10m"],"image":"busybox","name":"netdevice-l3-pod","ports":
[{"containerPort":80}]}],"restartPolicy":"Always"}}
    networking.gke.io/default-interface: eth0
    networking.gke.io/interfaces: |
      [
        {"interfaceName":"eth0","network":"default"},
        {"interfaceName":"eth1","network":"l3-network"},
        {"interfaceName":"eth2","network":"netdevice-network"}
      ]
    networking.gke.io/pod-ips: '[{"networkName":"l3-network","ip":"10.0.8.4"},
{"networkName":"netdevice-network","ip":"192.168.10.2"}]'
  creationTimestamp: "2023-07-30T22:49:27Z"
  name: netdevice-l3-pod
  namespace: default
  resourceVersion: "1585567"
  uid: d9e43c75-e0d1-4f31-91b0-129bc53bbf64
spec:
  containers:
    - command:
        - sleep
        - 10m
      image: busybox
      imagePullPolicy: Always
      name: netdevice-l3-pod
      ports:
        - containerPort: 80
          protocol: TCP
      resources:
        limits:
          networking.gke.io/networks/l3-network.IP: "1"
          networking.gke.io/networks/netdevice-network: "1"
          networking.gke.io/networks/netdevice-network.IP: "1"
        requests:
          networking.gke.io/networks/l3-network.IP: "1"
          networking.gke.io/networks/netdevice-network: "1"
          networking.gke.io/networks/netdevice-network.IP: "1"
      terminationMessagePath: /dev/termination-log
      terminationMessagePolicy: File
  volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: kube-api-access-f2wpb

```



```
    readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: gke-multinic-gke-multinic-node-pool-135699a1-86gz
  preemptionPolicy: PreemptLowerPriority
  priority: 0
  restartPolicy: Always
  schedulerName: default-scheduler
  securityContext: {}
  serviceAccount: default
  serviceAccountName: default
  terminationGracePeriodSeconds: 30
  tolerations:
  - effect: NoExecute
    key: node.kubernetes.io/not-ready
    operator: Exists
    tolerationSeconds: 300
  - effect: NoExecute
    key: node.kubernetes.io/unreachable
    operator: Exists
    tolerationSeconds: 300
  - effect: NoSchedule
    key: networking.gke.io/networks/l3-network.IP
    operator: Exists
  - effect: NoSchedule
    key: networking.gke.io/networks/netdevice-network
    operator: Exists
  - effect: NoSchedule
    key: networking.gke.io/networks/netdevice-network.IP
    operator: Exists
  volumes:
  - name: kube-api-access-f2wpb
    projected:
      defaultMode: 420
      sources:
      - serviceAccountToken:
          expirationSeconds: 3607
          path: token
      - configMap:
          items:
          - key: ca.crt
            path: ca.crt
            name: kube-root-ca.crt
      - downwardAPI:
          items:
          - fieldRef:
              apiVersion: v1
              fieldPath: metadata.namespace
            path: namespace
  status:
    conditions:
    - lastProbeTime: null
```

```

    lastTransitionTime: "2023-07-30T22:49:28Z"
    status: "True"
    type: Initialized
- lastProbeTime: null
  lastTransitionTime: "2023-07-30T22:49:33Z"
  status: "True"
  type: Ready
- lastProbeTime: null
  lastTransitionTime: "2023-07-30T22:49:33Z"
  status: "True"
  type: ContainersReady
- lastProbeTime: null
  lastTransitionTime: "2023-07-30T22:49:28Z"
  status: "True"
  type: PodScheduled
containerStatuses:
- containerID:
containerd://dcd9ead2f69824ccc37c109a47b1f3f5eb7b3e60ce3865e317dd729685b66a5c
  image: docker.io/library/busybox:latest
  imageID:
docker.io/library/busybox@sha256:3fbc632167424a6d997e74f52b878d7cc478225cffac6bc977eedfe51c7f4e79
  lastState: {}
  name: netdevice-l3-pod
  ready: true
  restartCount: 0
  started: true
  state:
    running:
      startedAt: "2023-07-30T22:49:32Z"
hostIP: 192.168.0.4
phase: Running
podIP: 10.0.1.22
podIPs:
- ip: 10.0.1.22
qosClass: BestEffort
startTime: "2023-07-30T22:49:28Z"

```

驗證 netdevice-l3-pod 路由

在 Cloud Shell 中，從 netdevice-l3-pod 驗證 netdevice-vpc 和 l3-vpc 的路徑：

```
kubectl exec --stdin --tty netdevice-l3-pod -- /bin/sh
```

建立執行個體，驗證 Pod 介面：

```
ifconfig
```

在本範例中，eth0 連接至預設網路，eth1 連接至 l3-network，eth2 則連接至 netdevice-network。

```
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 26:E3:1B:14:6E:0C
          inet addr:10.0.1.22  Bcast:10.0.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1460  Metric:1
          RX packets:5 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:446 (446.0 B)  TX bytes:558 (558.0 B)

eth1      Link encap:Ethernet  HWaddr 92:78:4E:CB:F2:D4
          inet addr:10.0.8.4  Bcast:0.0.0.0  Mask:255.255.255.255
          UP BROADCAST RUNNING MULTICAST  MTU:1460  Metric:1
          RX packets:5 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:446 (446.0 B)  TX bytes:516 (516.0 B)

eth2      Link encap:Ethernet  HWaddr 42:01:C0:A8:0A:02
          inet addr:192.168.10.2  Bcast:0.0.0.0  Mask:255.255.255.255
          UP BROADCAST RUNNING MULTICAST  MTU:1460  Metric:1
          RX packets:73 errors:0 dropped:0 overruns:0 frame:0
          TX packets:50581 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:26169 (25.5 KiB)  TX bytes:2148170 (2.0 MiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

從 netdevice-l3-pod 驗證前往 netdevice-vpc (172.16.10.0/28) 和 l3-vpc (172.16.20.0/28) 的路徑。

建立執行個體，驗證 Pod 路徑：

```
ip route
```

範例：

```
/ # ip route
default via 10.0.1.1 dev eth0 #primary-vpc
10.0.1.0/24 via 10.0.1.1 dev eth0 src 10.0.1.22
10.0.1.1 dev eth0 scope link src 10.0.1.22
10.0.8.0/21 via 10.0.8.1 dev eth1 #l3-vpc (sec-range-l3-subnet)
10.0.8.1 dev eth1 scope link
172.16.10.0/28 via 192.168.10.1 dev eth2 #netdevice-vpc (netdevice-apache subnet)
172.16.20.0/28 via 10.0.8.1 dev eth1 #l3-vpc (l3-apache subnet)
192.168.10.0/24 via 192.168.10.1 dev eth2 #pod interface subnet
192.168.10.1 dev eth2 scope link
```

如要返回 Cloud Shell，請從執行個體退出 Pod。

```
exit
```

步驟 12 · 約 5 分鐘

12. 建立 I3-pod

在下一節中，您將建立執行 busybox 的 I3-pod，這類 pod 支援超過 300 個常見指令，因此又稱為「瑞士刀」。Pod 設定為僅使用 eth1 與 I3-vpc 通訊。

在 Cloud Shell 中，使用 VI 編輯器或 nano 建立名為 I3-pod.yaml 的 BusyBox 容器。

```
apiVersion: v1
kind: Pod
metadata:
  name: i3-pod
  annotations:
    networking.gke.io/default-interface: 'eth0'
    networking.gke.io/interfaces: |
      [
        {"interfaceName":"eth0","network":"default"},
        {"interfaceName":"eth1","network":"i3-network"}
      ]
spec:
  containers:
    - name: i3-pod
      image: busybox
      command: ["sleep", "10m"]
      ports:
        - containerPort: 80
      restartPolicy: Always
```

在 Cloud Shell 中套用 I3-pod.yaml

```
kubectl apply -f i3-pod.yaml
```

驗證 I3-pod 建立作業

在 Cloud Shell 中，驗證 netdevice-i3-pod 是否正在執行：

```
kubectl get pods i3-pod
```

範例：

```
user@$ kubectl get pods i3-pod
NAME      READY   STATUS    RESTARTS   AGE
i3-pod    1/1     Running   0           52s
```

在 Cloud Shell 中，驗證指派給 Pod 介面的 IP 位址。

```
kubectl get pods i3-pod -o yaml
```

在提供的範例中，networking.gke.io/pod-ips 欄位包含與 I3 網路中 Pod 介面相關聯的 IP 位址。預設網路 IP 位址 10.0.2.12 的詳細資料位於 podIPs 下方：

```

user@$ kubectl get pods l3-pod -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"networking.gke.io/default-
interface":"eth0","networking.gke.io/interfaces":""
[\n{"interfaceName":"eth0","network":"default"},\n{"interfaceName":"eth1","netwo
rk":"l3-network"}\n]\n},"name":"l3-pod","namespace":"default"},"spec":{"containers":
[{"command":["sleep","10m"],"image":"busybox","name":"l3-pod","ports":
[{"containerPort":80}]}],"restartPolicy":"Always"}}
    networking.gke.io/default-interface: eth0
    networking.gke.io/interfaces: |
      [
        {"interfaceName":"eth0","network":"default"},
        {"interfaceName":"eth1","network":"l3-network"}
      ]
    networking.gke.io/pod-ips: '[{"networkName":"l3-network","ip":"10.0.8.22"}]'
creationTimestamp: "2023-07-30T23:22:29Z"
name: l3-pod
namespace: default
resourceVersion: "1604447"
uid: 79a86afd-2a50-433d-9d48-367acb82c1d0
spec:
  containers:
    - command:
        - sleep
        - 10m
      image: busybox
      imagePullPolicy: Always
      name: l3-pod
      ports:
        - containerPort: 80
          protocol: TCP
      resources:
        limits:
          networking.gke.io/networks/l3-network.IP: "1"
        requests:
          networking.gke.io/networks/l3-network.IP: "1"
      terminationMessagePath: /dev/termination-log
      terminationMessagePolicy: File
      volumeMounts:
        - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
          name: kube-api-access-w9d24
          readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: gke-multinic-gke-multinic-node-pool-135699a1-t66p
  preemptionPolicy: PreemptLowerPriority
  priority: 0
  restartPolicy: Always

```

```
schedulerName: default-scheduler
securityContext: {}
serviceAccount: default
serviceAccountName: default
terminationGracePeriodSeconds: 30
tolerations:
- effect: NoExecute
  key: node.kubernetes.io/not-ready
  operator: Exists
  tolerationSeconds: 300
- effect: NoExecute
  key: node.kubernetes.io/unreachable
  operator: Exists
  tolerationSeconds: 300
- effect: NoSchedule
  key: networking.gke.io/networks/l3-network.IP
  operator: Exists
volumes:
- name: kube-api-access-w9d24
  projected:
    defaultMode: 420
    sources:
    - serviceAccountToken:
        expirationSeconds: 3607
        path: token
    - configMap:
        items:
        - key: ca.crt
          path: ca.crt
        name: kube-root-ca.crt
    - downwardAPI:
        items:
        - fieldRef:
            apiVersion: v1
            fieldPath: metadata.namespace
            path: namespace
status:
  conditions:
  - lastProbeTime: null
    lastTransitionTime: "2023-07-30T23:22:29Z"
    status: "True"
    type: Initialized
  - lastProbeTime: null
    lastTransitionTime: "2023-07-30T23:22:35Z"
    status: "True"
    type: Ready
  - lastProbeTime: null
    lastTransitionTime: "2023-07-30T23:22:35Z"
    status: "True"
    type: ContainersReady
  - lastProbeTime: null
    lastTransitionTime: "2023-07-30T23:22:29Z"
```



```
    status: "True"
    type: PodScheduled
  containerStatuses:
  - containerID:
    containerd://1d5fe2854bba0a0d955c157a58bcfd4e34cecf8837edfd7df2760134f869e966
    image: docker.io/library/busybox:latest
    imageID:
    docker.io/library/busybox@sha256:3fbc632167424a6d997e74f52b878d7cc478225cffac6bc977eedfe51c7f4e79
    lastState: {}
    name: l3-pod
    ready: true
    restartCount: 0
    started: true
    state:
      running:
        startedAt: "2023-07-30T23:22:35Z"
  hostIP: 192.168.0.5
  phase: Running
  podIP: 10.0.2.12
  podIPs:
  - ip: 10.0.2.12
  qosClass: BestEffort
  startTime: "2023-07-30T23:22:29Z"
```

驗證 l3-pod 路由

在 Cloud Shell 中，從 netdevice-l3-pod 驗證 l3-vpc 的路徑：

```
kubectl exec --stdin --tty l3-pod -- /bin/sh
```

建立執行個體，驗證 Pod 介面：

```
ifconfig
```

在本範例中，eth0 連接至預設網路，eth1 連接至 l3 網路。

```
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 22:29:30:09:6B:58
          inet addr:10.0.2.12  Bcast:10.0.2.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1460  Metric:1
          RX packets:5 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:446 (446.0 B)  TX bytes:558 (558.0 B)

eth1      Link encap:Ethernet  HWaddr 6E:6D:FC:C3:FF:AF
          inet addr:10.0.8.22  Bcast:0.0.0.0  Mask:255.255.255.255
          UP BROADCAST RUNNING MULTICAST  MTU:1460  Metric:1
          RX packets:5 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:446 (446.0 B)  TX bytes:516 (516.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

從 l3-pod 驗證 l3-vpc (172.16.20.0/28) 的路徑。

建立執行個體，驗證 Pod 路徑：

```
ip route
```

範例：

```
/ # ip route
default via 10.0.2.1 dev eth0 #primary-vpc
10.0.2.0/24 via 10.0.2.1 dev eth0 src 10.0.2.12
10.0.2.1 dev eth0 scope link src 10.0.2.12
10.0.8.0/21 via 10.0.8.17 dev eth1 #l3-vpc (sec-range-l3-subnet)
10.0.8.17 dev eth1 scope link #pod interface subnet
172.16.20.0/28 via 10.0.8.17 dev eth1 #l3-vpc (l3-apache subnet)
```

如要返回 Cloud Shell，請從執行個體退出 Pod。

```
exit
```

13. 防火牆更新

如要允許從 GKE 多重 NIC 集區連線至 netdevice-vpc，必須設定 l3-vpc 輸入防火牆規則。您將建立防火牆規則，並將來源範圍指定為 Pod 網路子網路，例如 netdevice-subnet、sec-range-l3-subnet。

舉例來說，連線至 l3-vpc 中的 l3-apache 執行個體時，最近建立的容器 l3-pod、eth2 介面 10.0.8.22 (從 sec-range-l3-subnet 分配) 是來源 IP 位址。

netdevice-vpc：允許從 netdevice-subnet 傳送至 netdevice-apache

在 Cloud Shell 中，於 netdevice-vpc 建立防火牆規則，允許 netdevice-subnet 存取 netdevice-apache 執行個體。

```
gcloud compute --project=$projectid firewall-rules create allow-ingress-from-netdevice-  
network-to-all-vpc-instances --direction=INGRESS --priority=1000 --network=netdevice-vpc --  
action=ALLOW --rules=all --source-ranges=192.168.10.0/24 --enable-logging
```

l3-vpc：允許從 sec-range-l3-subnet 到 l3-apache

在 Cloud Shell 中，於 l3-vpc 建立防火牆規則，允許 sec-range-l3-subnet 存取 l3-apache 執行個體。

```
gcloud compute --project=$projectid firewall-rules create allow-ingress-from-l3-network-to-  
all-vpc-instances --direction=INGRESS --priority=1000 --network=l3-vpc --action=ALLOW --  
rules=all --source-ranges=10.0.8.0/21 --enable-logging
```

14. 驗證 Pod 連線

在下一節中，您將登入 Pod 並執行 `wget -S`，驗證 Apache 伺服器首頁的下載作業，藉此驗證 `netdevice-l3-pod` 和 `l3-pod` 與 Apache 執行個體的連線。由於 `netdevice-l3-pod` 是透過 `netdevice-network` 和 `l3-network` 的介面設定，因此可以連線至 `netdevice-vpc` 和 `l3-vpc` 中的 Apache 伺服器。

相反地，從 `l3-pod` 執行 `wget -S` 時，由於 `l3-pod` 只設定了 `l3-network` 的介面，因此無法連線至 `netdevice-vpc` 中的 Apache 伺服器。

取得 Apache 伺服器 IP 位址

從 Cloud 控制台前往「Compute Engine」→「VM 執行個體」，取得 Apache 伺服器的 IP 位址

INSTANCES OBSERVABILITY INSTANCE SCHEDULES								
VM instances								
Filter Enter property name or value								
☐	Status	Name ↑	Zone	Recommendations	In use by	Internal IP	External IP	Network
☐	✓	gke-multinic-gke-default-pool-3d419e48-1k2p	us-central1-a		gke-multinic-gke-default-p...	192.168.0.2 (nic0)		primary-vpc
☐	✓	gke-multinic-gke-default-pool-3d419e48-xckb	us-central1-a		gke-multinic-gke-default-p...	192.168.0.3 (nic0)		primary-vpc
☐	✓	gke-multinic-gke-multinic-node-pool-135699a1-0tfx	us-central1-a		gke-multinic-gke-multinic...	192.168.0.6 (nic0) 192.168.10.4 (nic1) 192.168.20.4 (nic2)		primary-vpc
☐	✓	gke-multinic-gke-multinic-node-pool-135699a1-86gz	us-central1-a		gke-multinic-gke-multinic...	192.168.0.4 (nic0) 192.168.10.2 (nic1) 192.168.20.2 (nic2)		primary-vpc
☐	✓	gke-multinic-gke-multinic-node-pool-135699a1-t66p	us-central1-a		gke-multinic-gke-multinic...	192.168.0.5 (nic0) 192.168.10.3 (nic1) 192.168.20.3 (nic2)		primary-vpc
☐	✓	l3-apache	us-central1-a			172.16.20.3 (nic0)		l3-vpc
☐	✓	netdevice-apache	us-central1-a			172.16.10.2 (nic0)		netdevice-vpc

netdevice-l3-pod 至 netdevice-apache 的連線能力測試

在 Cloud Shell 中，登入 `netdevice-l3-pod`：

```
kubectl exec --stdin --tty netdevice-l3-pod -- /bin/sh
```

從容器中，根據上一步驟取得的 IP 位址，對 `netdevice-apache` 執行個體執行 `ping`。

```
ping <insert-your-ip> -c 4
```

範例：

```
/ # ping 172.16.10.2 -c 4
PING 172.16.10.2 (172.16.10.2): 56 data bytes
64 bytes from 172.16.10.2: seq=0 ttl=64 time=1.952 ms
64 bytes from 172.16.10.2: seq=1 ttl=64 time=0.471 ms
64 bytes from 172.16.10.2: seq=2 ttl=64 time=0.446 ms
64 bytes from 172.16.10.2: seq=3 ttl=64 time=0.505 ms

--- 172.16.10.2 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.446/0.843/1.952 ms
/ #
```

在 Cloud Shell 中，根據上一個步驟取得的 IP 位址，對 netdevice-apache 執行個體執行 `wget -S`，200 OK 表示網頁下載成功。

```
wget -S <insert-your-ip>
```

範例：

```
/ # wget -S 172.16.10.2
Connecting to 172.16.10.2 (172.16.10.2:80)
  HTTP/1.1 200 OK
  Date: Mon, 31 Jul 2023 03:12:58 GMT
  Server: Apache/2.4.56 (Debian)
  Last-Modified: Sat, 29 Jul 2023 00:32:44 GMT
  ETag: "2c-6019555f54266"
  Accept-Ranges: bytes
  Content-Length: 44
  Connection: close
  Content-Type: text/html

saving to 'index.html'
index.html          100% |*****|          44  0:00:00 ETA
'index.html' saved
/ #
```

netdevice-I3-pod 至 I3-apache 連線測試

在 Cloud Shell 中，根據上一個步驟取得的 IP 位址，對 I3-apache 執行個體執行 `ping`。

```
ping <insert-your-ip> -c 4
```

範例：

```
/ # ping 172.16.20.3 -c 4
PING 172.16.20.3 (172.16.20.3): 56 data bytes
64 bytes from 172.16.20.3: seq=0 ttl=63 time=2.059 ms
64 bytes from 172.16.20.3: seq=1 ttl=63 time=0.533 ms
64 bytes from 172.16.20.3: seq=2 ttl=63 time=0.485 ms
64 bytes from 172.16.20.3: seq=3 ttl=63 time=0.462 ms

--- 172.16.20.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.462/0.884/2.059 ms
/ #
```

在 Cloud Shell 中，刪除先前的 `index.html` 檔案，並根據上一個步驟取得的 IP 位址，對 I3-apache 執行個體執行 `wget -S`，200 OK 表示網頁已成功下載。

```
rm index.html
wget -S <insert-your-ip>
```

範例：

```
/ # rm index.html
/ # wget -S 172.16.20.3
Connecting to 172.16.20.3 (172.16.20.3:80)
  HTTP/1.1 200 OK
  Date: Mon, 31 Jul 2023 03:41:32 GMT
  Server: Apache/2.4.56 (Debian)
  Last-Modified: Mon, 31 Jul 2023 03:24:21 GMT
  ETag: "25-601bff76f04b7"
  Accept-Ranges: bytes
  Content-Length: 37
  Connection: close
  Content-Type: text/html

saving to 'index.html'
index.html           100%
|*****|
*****|      37   0:00:00 ETA
'index.html' saved
```

如要返回 Cloud Shell，請從執行個體退出 Pod。

```
exit
```

I3-pod 至 netdevice-apache 連線測試

在 Cloud Shell 中，登入 I3-pod：

```
kubectl exec --stdin --tty i3-pod -- /bin/sh
```

從容器對 netdevice-apache 執行個體執行 ping，依據是上一步驟取得的 IP 位址。由於 I3-pod 沒有與 netdevice-network 相關聯的介面，因此連線偵測 (ping) 會失敗。

```
ping <insert-your-ip> -c 4
```

範例：

```
/ # ping 172.16.10.2 -c 4
PING 172.16.10.2 (172.16.10.2): 56 data bytes

--- 172.16.10.2 ping statistics ---
4 packets transmitted, 0 packets received, 100% packet loss
```

選用：在 Cloud Shell 中，根據上一個步驟取得的 IP 位址，對 netdevice-apache 執行個體執行 `wget -S`，該執行個體會逾時。

```
wget -S <insert-your-ip>
```

範例：

```
/ # wget -S 172.16.10.2
Connecting to 172.16.10.2 (172.16.10.2:80)
wget: can't connect to remote host (172.16.10.2): Connection timed out
```

I3-pod 至 I3-apache 連線測試

在 Cloud Shell 中，根據上一個步驟取得的 IP 位址，對 I3-apache 執行個體執行 `ping`。

```
ping <insert-your-ip> -c 4
```

範例：

```
/ # ping 172.16.20.3 -c 4
PING 172.16.20.3 (172.16.20.3): 56 data bytes
64 bytes from 172.16.20.3: seq=0 ttl=63 time=1.824 ms
64 bytes from 172.16.20.3: seq=1 ttl=63 time=0.513 ms
64 bytes from 172.16.20.3: seq=2 ttl=63 time=0.482 ms
64 bytes from 172.16.20.3: seq=3 ttl=63 time=0.532 ms

--- 172.16.20.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.482/0.837/1.824 ms
/ #
```

在 Cloud Shell 中，根據上一個步驟取得的 IP 位址，對 I3-apache 執行個體執行 `wget -S`，200 OK 表示網頁下載成功。

```
wget -S <insert-your-ip>
```

範例：

```
/ # wget -S 172.16.20.3
Connecting to 172.16.20.3 (172.16.20.3:80)
  HTTP/1.1 200 OK
  Date: Mon, 31 Jul 2023 03:52:08 GMT
  Server: Apache/2.4.56 (Debian)
  Last-Modified: Mon, 31 Jul 2023 03:24:21 GMT
  ETag: "25-601bff76f04b7"
  Accept-Ranges: bytes
  Content-Length: 37
  Connection: close
  Content-Type: text/html

saving to 'index.html'
index.html          100%
|*****
*****|      37  0:00:00 ETA
'index.html' saved
/ #
```

步驟 15 · 約 5 分鐘

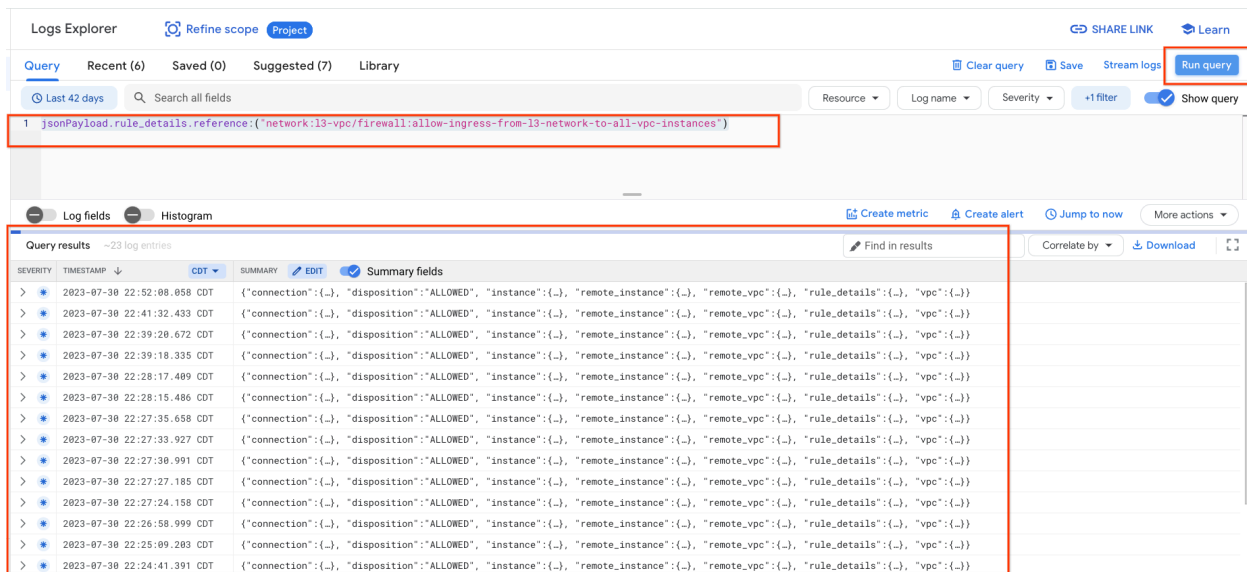
15. 防火牆記錄

您可以根據防火牆規則記錄，進一步稽核、驗證及分析防火牆規則的成效。舉例來說，您可以判斷用於拒絕流量的防火牆規則是否正常運作。需要瞭解特定防火牆規則影響的連線數量時，這項功能也能派上用場。

在本教學課程中，您在建立輸入防火牆規則時已啟用防火牆記錄。我們來看看從記錄中取得的資訊。

在 Cloud 控制台中，依序前往「Logging」→「Logs Explorer」

按照螢幕截圖插入下列查詢，然後選取「Run query jsonPayload.rule_details.reference:("network:l3-vpc/firewall:allow-ingress-from-l3-network-to-all-vpc-instances")」。



深入瞭解擷取內容，即可取得安全管理員所需的資訊元素，包括來源和目的地 IP 位址、通訊埠、通訊協定和節點集區名稱。

```
{
  insertId: "6dwa7bf81km72"
  jsonPayload: {
    connection: {
      dest_ip: "172.16.20.3"
      dest_port: 80
      protocol: 6
      src_ip: "10.0.8.22"
      src_port: 55158
    }
    disposition: "ALLOWED"
    instance: {
      project_id: "..."
      region: "us-central1"
      vm_name: "l3-apache"
      zone: "us-central1-a"
    }
    remote_instance: {
      project_id: "..."
      region: "us-central1"
      vm_name: "gke-multinic-gke-multinic-node-pool-135699a1-t66p"
      zone: "us-central1-a"
    }
  }
}
```

如要進一步探索防火牆記錄，請依序前往「虛擬私有雲網路」→「防火牆」→「allow-ingress-from-netdevice-network-to-all-vpc-instances」，然後選取「在 Logs Explorer 中查看」。

16. 清理

在 Cloud Shell 中刪除教學課程元件。

```
gcloud compute instances delete l3-apache netdevice-apache --zone=us-central1-a --quiet

gcloud compute routers delete l3-cr netdevice-cr --region=us-central1 --quiet

gcloud container clusters delete multinic-gke --zone=us-central1-a --quiet

gcloud compute firewall-rules delete allow-ingress-from-l3-network-to-all-vpc-instances
allow-ingress-from-netdevice-network-to-all-vpc-instances --quiet

gcloud compute networks subnets delete l3-apache l3-subnet netdevice-apache netdevice-subnet
primary-node-subnet --region=us-central1 --quiet

gcloud compute networks delete l3-vpc netdevice-vpc primary-vpc --quiet
```

步驟 17 · 約 0 分鐘

17. 恭喜

恭喜！您已成功設定並驗證建立多 NIC 節點集區，以及建立執行 busybox 的 Pod，使用 PING 和 wget 驗證與 Apache 伺服器的 L3 和裝置類型連線。

您也學到如何運用防火牆記錄，檢查 Pod 容器與 Apache 伺服器之間的來源和目的地封包。

Cosmopup 認為教學課程很棒！



延伸閱讀和影片

- [建立 GKE 叢集 \(示範\)](#)
- [GKE：網路的概念](#)
- [容器與 Kubernetes 網誌](#)

參考文件

- [GKE 網路](#)
 - [防火牆規則記錄](#)
-